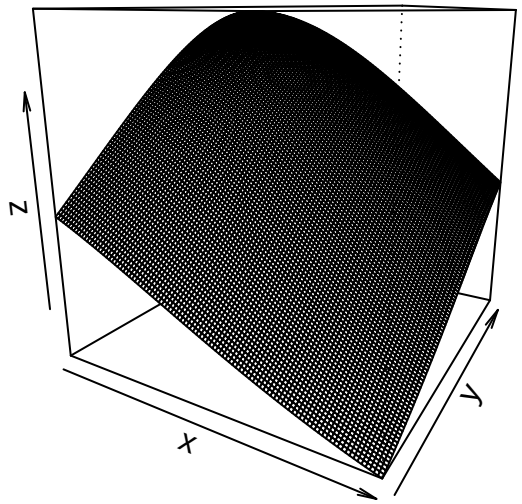
MS&ML Revision

The probability density function of a multvariate normal distrbution is defined as

$$f_x(x) = \frac{1}{(2\pi \det(\Sigma))^d} \exp(-\frac{1}{2}(x-\mu)^T \Sigma (x-\mu))$$

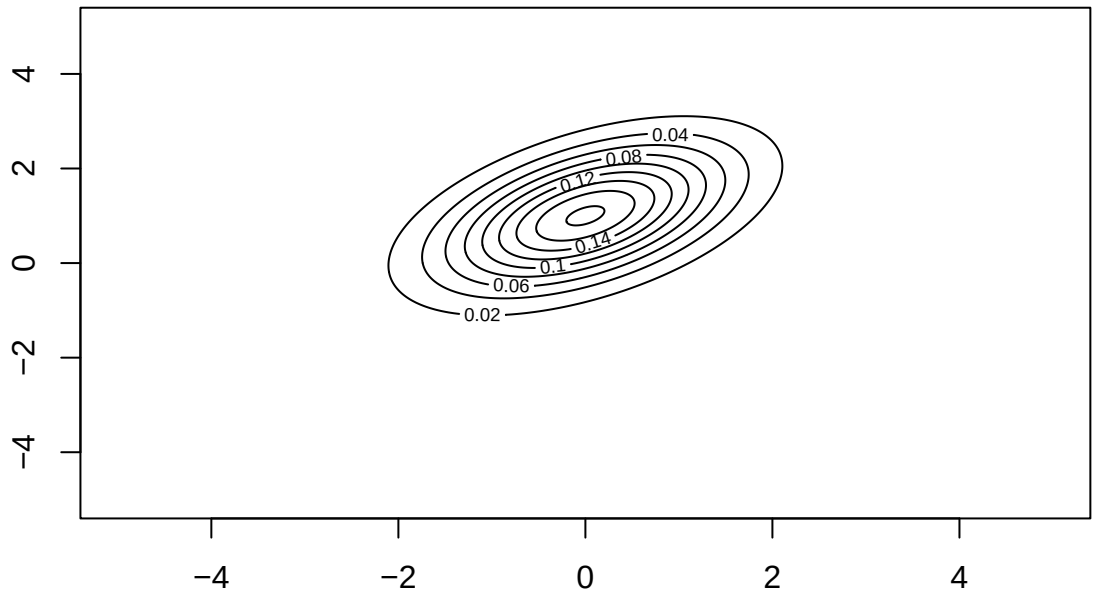
Example 1

```
library("mnormt")
mu=c(0,1)
Sigma=matrix(c(1,0.5,0.5,1),nrow=2)
base1<-seq(0,1,by=0.01)
set<-expand.grid(base1,base1)
temp<-dmnorm(set,mean=mu,varcov=Sigma)
temp2<-matrix(temp,nrow=length(base1))
persp(base1,base1,temp2,theta=30,
xlab="x",ylab="y",zlab="z",main="pdf of a multivariate normal distrbution")
```




```
base1<-seq(-5,5,by=0.01)
set<-expand.grid(base1,base1)
```

```
temp<-dmnorm(set,mean=mu,varcov=Sigma)
temp2<-matrix(temp,nrow=length(base1))
contour(base1,base1,temp2,xlim=c(-5,5),ylim=c(-5,5))
```




```
eigen(Sigma)
```

```
## eigen() decomposition
## $values
## [1] 1.5 0.5
##
## $vectors
##           [,1]       [,2]
## [1,] 0.7071068 -0.7071068
## [2,] 0.7071068  0.7071068
```

```
val<- eigen(Sigma)$values
vec<- eigen(Sigma)$vectors
temp1<-val[1]
temp2<-val[2]
sqrt(temp1/temp2);sqrt(temp2/temp1)
```

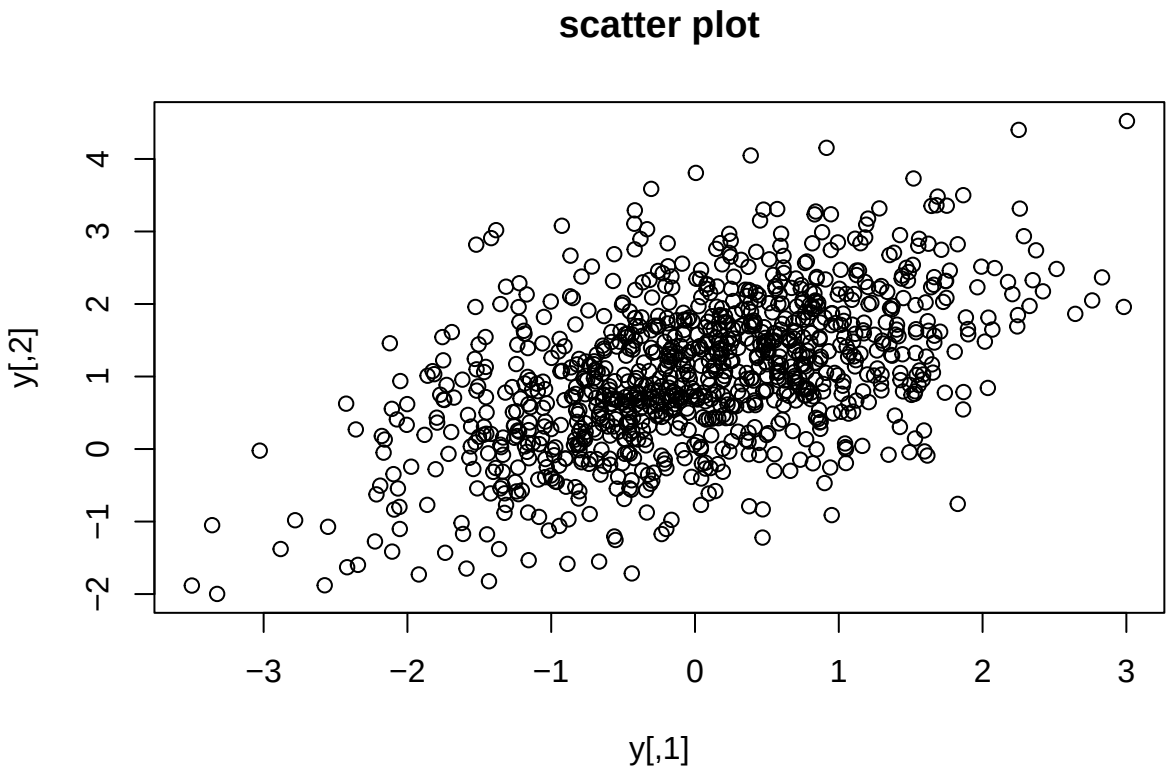
```
## [1] 1.732051
```

```
## [1] 0.5773503
```

```
y<-rmnorm(1000,mean=mu,varcov=Sigma)
head(y)
```

```
##             [,1]       [,2]
## [1,] -0.41075445  1.6945494
## [2,] -0.07791138  1.7846939
## [3,] -1.01659221 -1.1237196
## [4,] -1.06116661  1.4561843
## [5,] -1.51873999  0.8052864
## [6,] -0.58432237  0.6268019
```

```
plot(y,main="scatter plot")
```



Example 2

```
g1<-qnorm(0.995,0,1)
print(g1)
```

```
## [1] 2.575829
```

```
temp<-matrix(0,nrow=50)
library("mnormt")
alpha<-0.995
for (i in 1:50){
temp[i]<-qchisq(alpha,i)
}
head(temp)
```

```
##          [,1]
## [1,]  7.879439
```

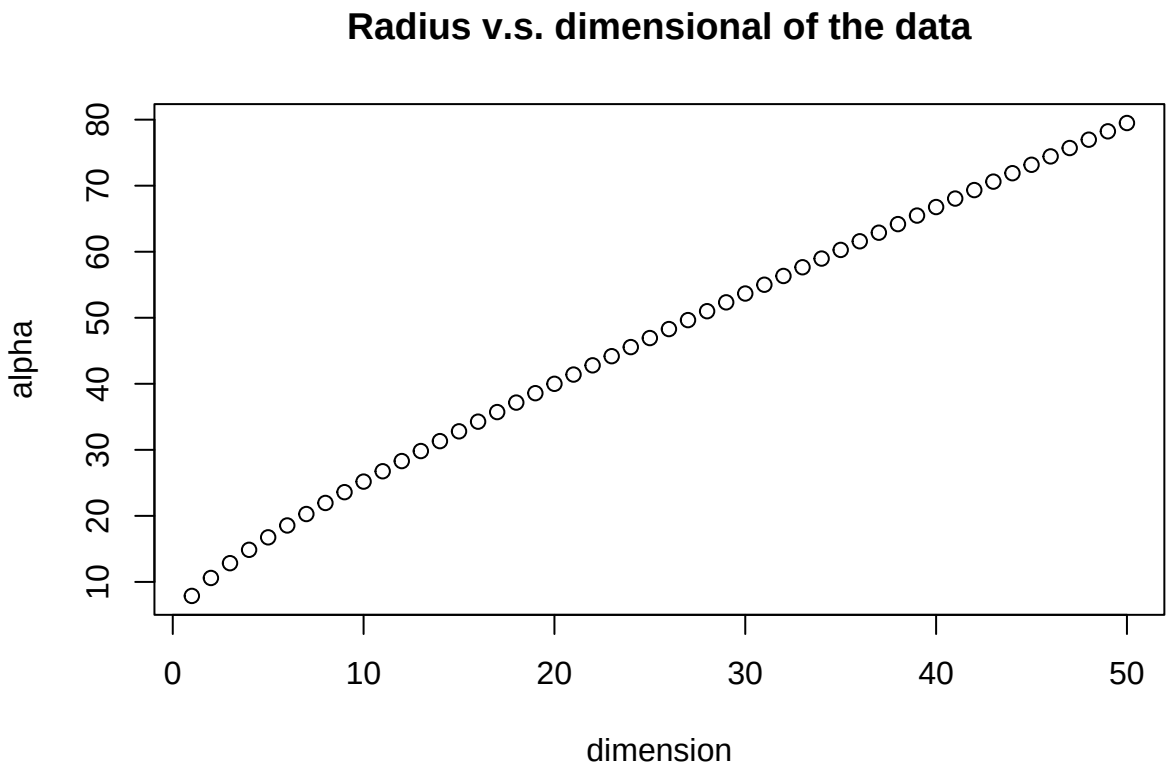
```
## [2,] 10.596635
## [3,] 12.838156
## [4,] 14.860259
## [5,] 16.749602
## [6,] 18.547584
```

```
base<-seq(1,50,by=1)
length(base);length(temp)
```

```
## [1] 50
```

```
## [1] 50
```

```
plot(base,temp,xlab="dimension",ylab="alpha",main="Radius v.s. dimensional of the data")
```

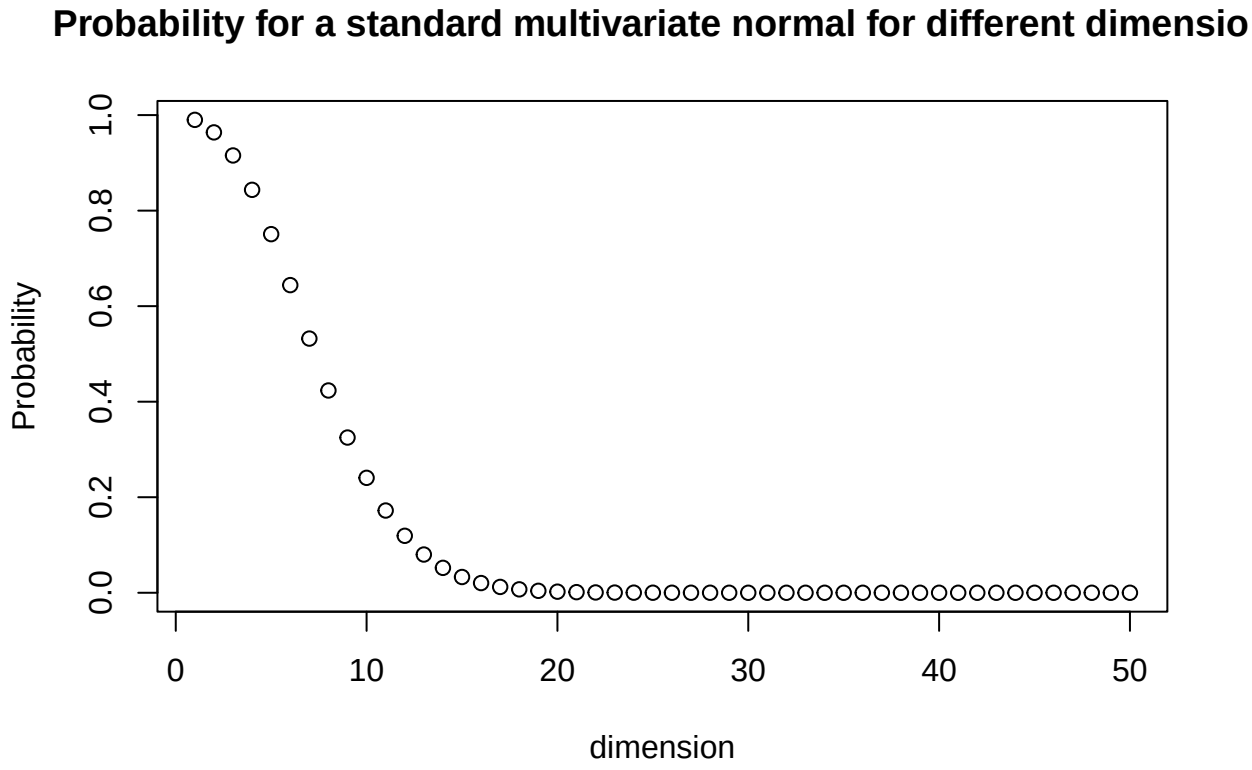



```
f0<-dnorm(0,0,1)
f1<-dnorm(g1,0,1)
thet<-f1/f0
print(thet)
```

```
## [1] 0.0362452
```

```
temp<-matrix(0,nrow=50)
for (i in 1:50){
temp[i]<- pchisq(-2*log(thet),i)
}
base<-seq(1,50,by=1)
plot(base,temp,
main="Probability for a standard multivariate normal for different dimension",
```

```
xlab="dimension",ylab="Probability")
```



Proof for Part 4:

Recall that

$$f_x(x) = \frac{1}{(2\pi \det(\Sigma))^d} \exp(-\frac{1}{2}(x-\mu)^T \Sigma (x-\mu))$$

Assume $X \sim MN(0, I_d)$

Then we have

$$f_x(x) = \frac{1}{(2\pi)^d} \exp(-\frac{1}{2}(x-\mu)^T (x-\mu))$$

$$f_x(0) = \frac{1}{(2\pi)^d}$$

$$\eta = \frac{f(x)}{f(0)} = \exp(-\frac{1}{2}(x^T x)) = exp(-\frac{1}{2} r_d^2)$$

$$r_d^2 = -2\log(\eta)$$

Example 3

```
conv<-diag(1,nrow=100,ncol=100)
temp<-eigen(conv)
temp$values
```

```
##   [1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
##  [38] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
##  [75] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
```

```
mu<-matrix(0,nrow=100,ncol=1)

set.seed(10)
par(mfrow=c(2,2))
for (i in c(20,50,200)){
x<-rmnorm(n=i,mean=mu,conv)
plot(x,xlab="x",ylab="density",
main=paste("Simulate data sets for n=",i))
}

S1.emp<-rmnorm(n=20,mean=mu,conv)
S2.emp<-rmnorm(n=50,mean=mu,conv)
S3.emp<-rmnorm(n=200,mean=mu,conv)
S1.eigen<-eigen(cov(S1.emp))$values
S2.eigen<-eigen(cov(S2.emp))$values
S3.eigen<-eigen(cov(S3.emp))$values

library("corpcor")
rank.condition(S1.emp)
```

```
## $rank
## [1] 20
##
## $condition
## [1] 2.512101
##
## $tol
## [1] 3.062711e-13
```

```
rank.condition(S2.emp)
```

```
## $rank
## [1] 50
##
## $condition
## [1] 5.113185
##
## $tol
## [1] 3.454298e-13
```

```
rank.condition(S3.emp)
```

```
## $rank
## [1] 100
##
## $condition
## [1] 4.952798
##
## $tol
## [1] 1.042117e-12
```

```
mean((S1.eigen-temp$values)^2)
```

```
## [1] 5.456978
```

```
mean((S2.eigen-temp$values)^2)
```

```
## [1] 1.86028
```

```
mean((S3.eigen-temp$values)^2)
```

```
## [1] 0.5050622
```

```
library("corpcor")
S1<- cov.shrink(S1.emp)
```

```
## Estimating optimal shrinkage intensity lambda.var (variance vector): 0.762
##
## Estimating optimal shrinkage intensity lambda (correlation matrix): 0.936
```

```
S2<- cov.shrink(S2.emp)
```

```
## Estimating optimal shrinkage intensity lambda.var (variance vector): 1
##
## Estimating optimal shrinkage intensity lambda (correlation matrix): 1
```

```
S3<- cov.shrink(S3.emp)
```

```
## Estimating optimal shrinkage intensity lambda.var (variance vector): 1
##
## Estimating optimal shrinkage intensity lambda (correlation matrix): 1
```

```
print(eigen(S1)$values)
```

```
##   [1] 1.4992130 1.4366102 1.4239778 1.3624452 1.3234785 1.3045087 1.2789860
##   [8] 1.2669234 1.2192288 1.2177591 1.1875742 1.1751748 1.1635281 1.1325020
##  [15] 1.1095411 1.0965554 1.0789279 1.0728981 1.0540942 1.0438360 1.0118037
##  [22] 1.0075486 1.0029732 0.9927427 0.9848157 0.9763136 0.9644458 0.9580034
##  [29] 0.9544312 0.9511112 0.9445423 0.9418718 0.9414812 0.9364516 0.9319850
##  [36] 0.9283719 0.9250370 0.9212023 0.9169335 0.9167531 0.9094973 0.9052332
##  [43] 0.9010243 0.9004186 0.8983577 0.8963392 0.8903951 0.8884473 0.8832217
##  [50] 0.8822399 0.8766876 0.8758741 0.8722542 0.8704021 0.8685637 0.8670295
##  [57] 0.8649460 0.8612417 0.8605646 0.8584379 0.8564430 0.8548364 0.8539141
##  [64] 0.8519191 0.8499476 0.8487198 0.8472229 0.8456519 0.8436913 0.8406438
##  [71] 0.8389892 0.8377506 0.8355562 0.8329595 0.8314002 0.8302927 0.8283678
##  [78] 0.8266530 0.8258911 0.8239050 0.8225285 0.8216597 0.8184625 0.8167297
##  [85] 0.8157678 0.8140238 0.8119209 0.8056137 0.8051912 0.8037194 0.8023069
##  [92] 0.8013105 0.7960224 0.7928399 0.7897710 0.7842342 0.7778783 0.7636002
##  [99] 0.7585036 0.7437333
```

```
print(eigen(S2)$values)
```

```
##   [1] 0.9365153 0.9365153 0.9365153 0.9365153 0.9365153 0.9365153 0.9365153
##   [8] 0.9365153 0.9365153 0.9365153 0.9365153 0.9365153 0.9365153 0.9365153
##  [15] 0.9365153 0.9365153 0.9365153 0.9365153 0.9365153 0.9365153 0.9365153
##  [22] 0.9365153 0.9365153 0.9365153 0.9365153 0.9365153 0.9365153 0.9365153
##  [29] 0.9365153 0.9365153 0.9365153 0.9365153 0.9365153 0.9365153 0.9365153
##  [36] 0.9365153 0.9365153 0.9365153 0.9365153 0.9365153 0.9365153 0.9365153
##  [43] 0.9365153 0.9365153 0.9365153 0.9365153 0.9365153 0.9365153 0.9365153
##  [50] 0.9365153 0.9365153 0.9365153 0.9365153 0.9365153 0.9365153 0.9365153
##  [57] 0.9365153 0.9365153 0.9365153 0.9365153 0.9365153 0.9365153 0.9365153
##  [64] 0.9365153 0.9365153 0.9365153 0.9365153 0.9365153 0.9365153 0.9365153
```

```
##  [71] 0.9365153 0.9365153 0.9365153 0.9365153 0.9365153 0.9365153 0.9365153
##  [78] 0.9365153 0.9365153 0.9365153 0.9365153 0.9365153 0.9365153 0.9365153
##  [85] 0.9365153 0.9365153 0.9365153 0.9365153 0.9365153 0.9365153 0.9365153
##  [92] 0.9365153 0.9365153 0.9365153 0.9365153 0.9365153 0.9365153 0.9365153
##  [99] 0.9365153 0.9365153
```

```
print(eigen(S3)$values)
```

```
##   [1] 1.004518 1.004518 1.004518 1.004518 1.004518 1.004518 1.004518 1.004518
##   [9] 1.004518 1.004518 1.004518 1.004518 1.004518 1.004518 1.004518 1.004518
##  [17] 1.004518 1.004518 1.004518 1.004518 1.004518 1.004518 1.004518 1.004518
##  [25] 1.004518 1.004518 1.004518 1.004518 1.004518 1.004518 1.004518 1.004518
##  [33] 1.004518 1.004518 1.004518 1.004518 1.004518 1.004518 1.004518 1.004518
##  [41] 1.004518 1.004518 1.004518 1.004518 1.004518 1.004518 1.004518 1.004518
##  [49] 1.004518 1.004518 1.004518 1.004518 1.004518 1.004518 1.004518 1.004518
##  [57] 1.004518 1.004518 1.004518 1.004518 1.004518 1.004518 1.004518 1.004518
##  [65] 1.004518 1.004518 1.004518 1.004518 1.004518 1.004518 1.004518 1.004518
##  [73] 1.004518 1.004518 1.004518 1.004518 1.004518 1.004518 1.004518 1.004518
##  [81] 1.004518 1.004518 1.004518 1.004518 1.004518 1.004518 1.004518 1.004518
##  [89] 1.004518 1.004518 1.004518 1.004518 1.004518 1.004518 1.004518 1.004518
##  [97] 1.004518 1.004518 1.004518 1.004518
```

```
rank.condition(S1)
```

```
## $rank
## [1] 100
##
## $condition
## [1] 2.015794
##
## $tol
## [1] 3.328922e-14
```

```
rank.condition(S2)
```

```
## $rank
## [1] 100
##
## $condition
## [1] 1
##
## $tol
## [1] 2.079482e-14
```

```
rank.condition(S3)
```

```
## $rank
## [1] 100
##
## $condition
## [1] 1
##
## $tol
## [1] 2.230479e-14
```

```
mean((S1.eigen-temp$values)^2)
```

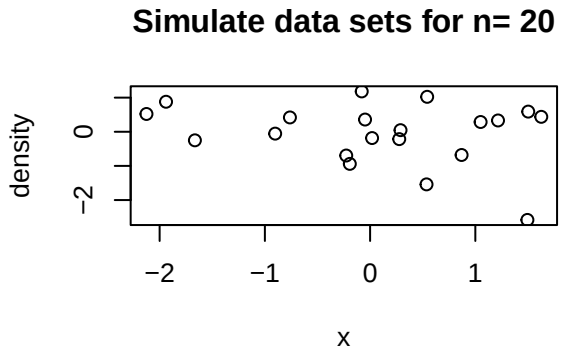
```
## [1] 5.456978
```

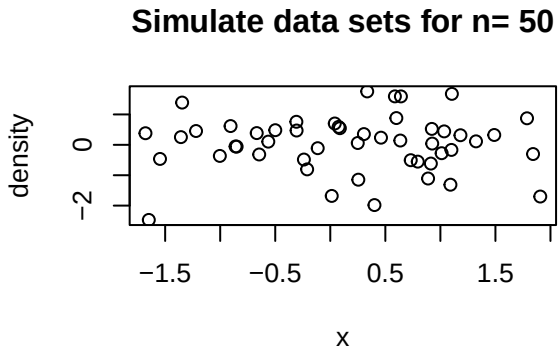
```r
mean((S2.eigen-temp$values)^2)
```

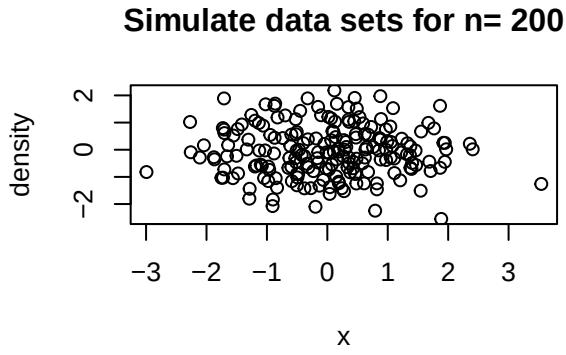
```
## [1] 1.86028
```

```r
mean((S3.eigen-temp$values)^2)
```

```
## [1] 0.5050622
```







Example 4

Set

$$\Sigma = \begin{pmatrix} 5 & 2 & 2 \\ 2 & 7 & 8 \\ 2 & 8 & 11 \end{pmatrix}$$

```r
mu=c(0,0,0)
Sigma<-matrix(c(5,2,2,2,7,8,2,8,11),nrow=3,ncol=3)
head(Sigma)
```

```
##      [,1] [,2] [,3]
## [1,]    5    2    2
## [2,]    2    7    8
## [3,]    2    8   11
```

```r
library("whitening")
library("mnormt")
data<-rmnorm(n=500,mean=mu,varcov=Sigma)
ZCA<- whiten(data,method="ZCA")
```

```
ZCAcor<- whiten(data,method="ZCA-cor")
PCA<- whiten(data,method="PCA")
PCAcor<- whiten(data,method="PCA-cor")
Chole<- whiten(data,method="Cholesky")

zapsmall(cov(ZCA))
```

```
##    L1 L2 L3
## L1  1  0  0
## L2  0  1  0
## L3  0  0  1
```

```
zapsmall(cov(ZCAcor))
```

```
##    L1 L2 L3
## L1  1  0  0
## L2  0  1  0
## L3  0  0  1
```

```
zapsmall(cov(PCA))
```

```
##    L1 L2 L3
## L1  1  0  0
## L2  0  1  0
## L3  0  0  1
```

```
zapsmall(cov(PCAcor))
```

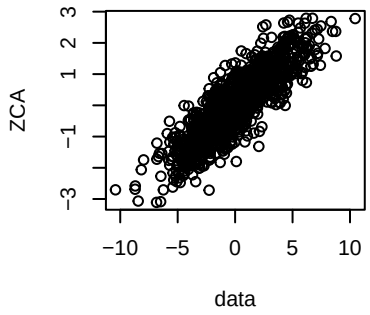
```
##    L1 L2 L3
## L1  1  0  0
## L2  0  1  0
## L3  0  0  1
```

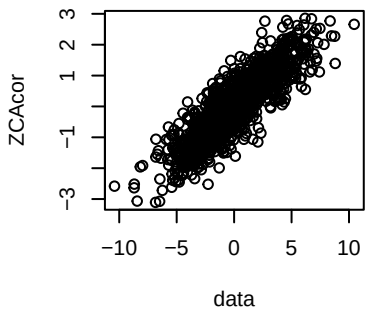
```
zapsmall(cov(Chole))
```

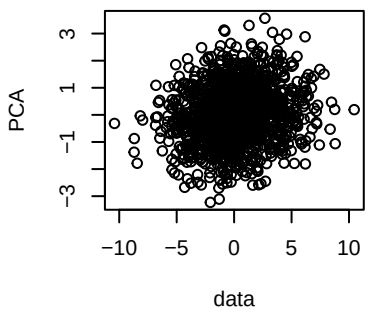
```
##    L1 L2 L3
## L1  1  0  0
## L2  0  1  0
## L3  0  0  1
```

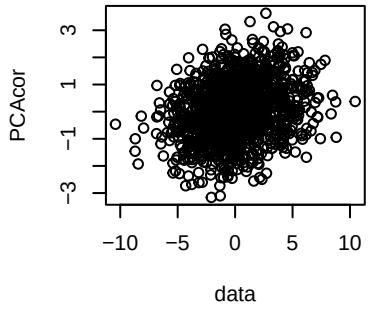
```
par(mfrow=c(2,3))
plot(data,ZCA)
plot(data,ZCAcor)
plot(data,PCA)
plot(data,PCAcor)
plot(data,Chole)

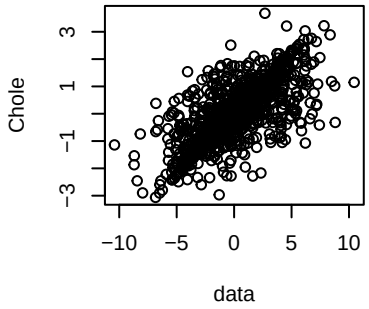
par(mfrow=c(2,3))
```







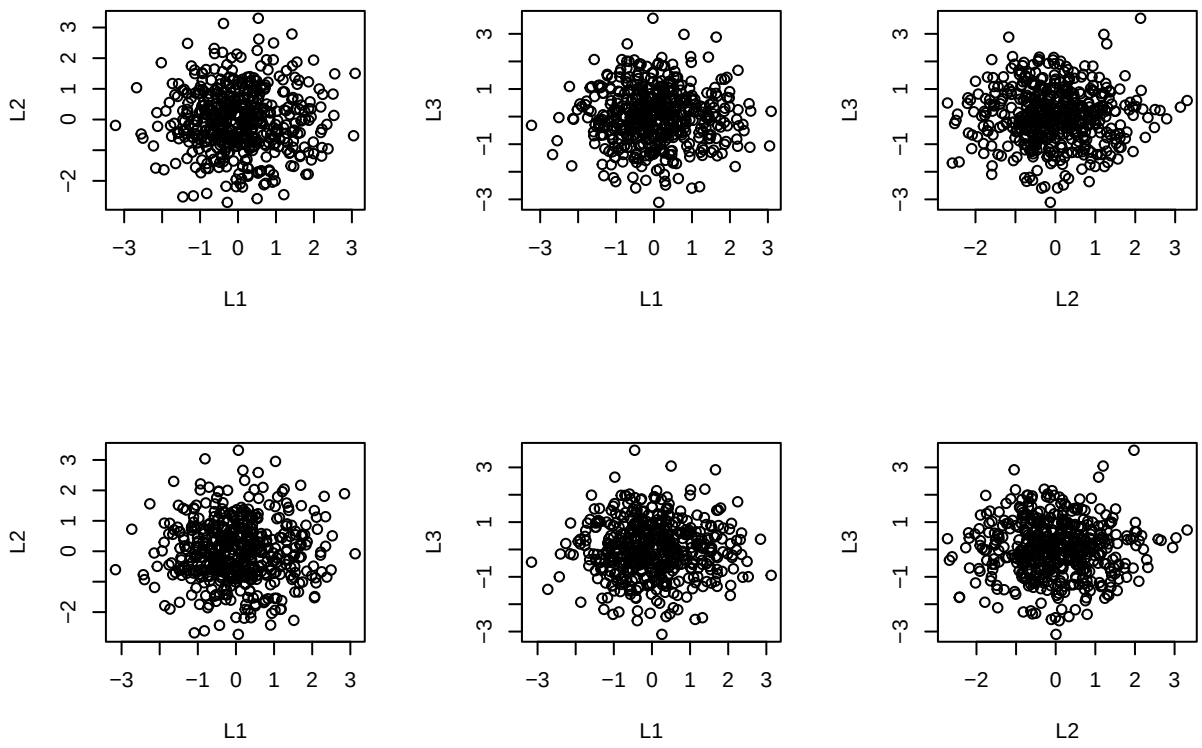




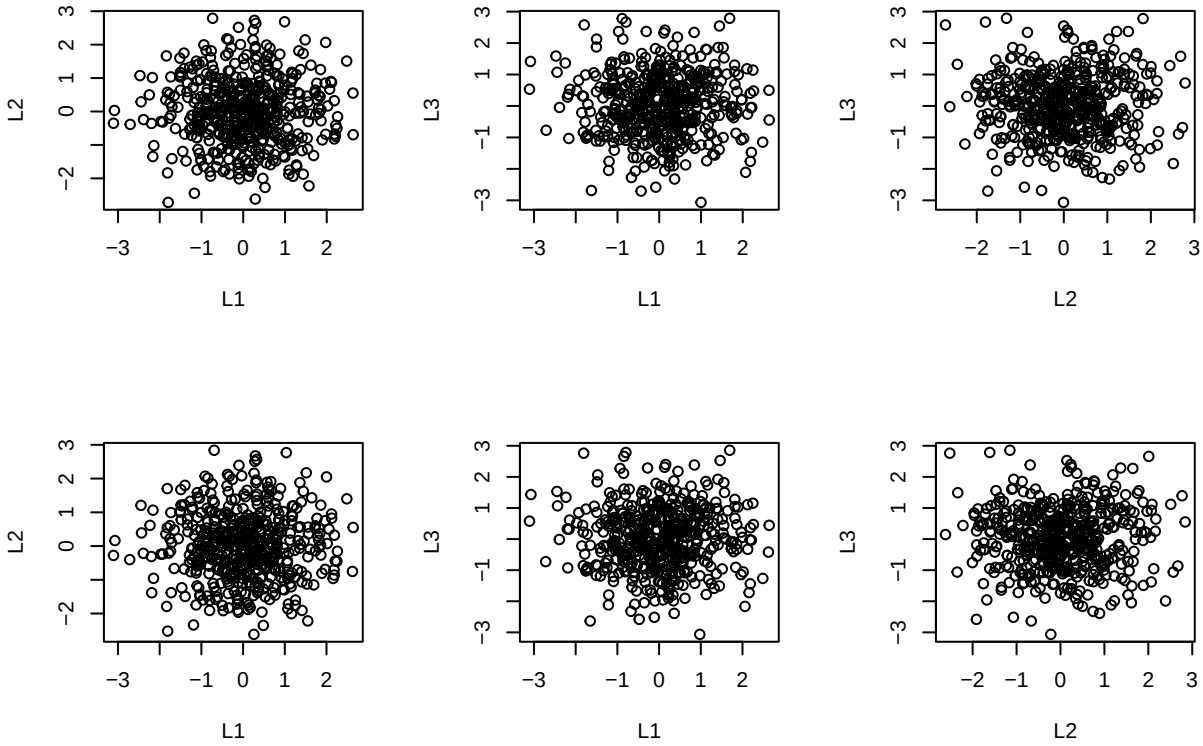

```
head(PCA)
```

```
##              L1           L2           L3
## [1,]  0.1492966 -1.077365572  1.093494511
## [2,]  0.9287391  1.946913109 -1.266880184
## [3,] -0.2201007  0.782940414 -0.834341358
## [4,]  1.7556652  0.268741746 -0.379325157
## [5,]  0.2125526 -0.820634638  1.847675147
## [6,] -0.3146445 -0.008900462 -0.009542811
```

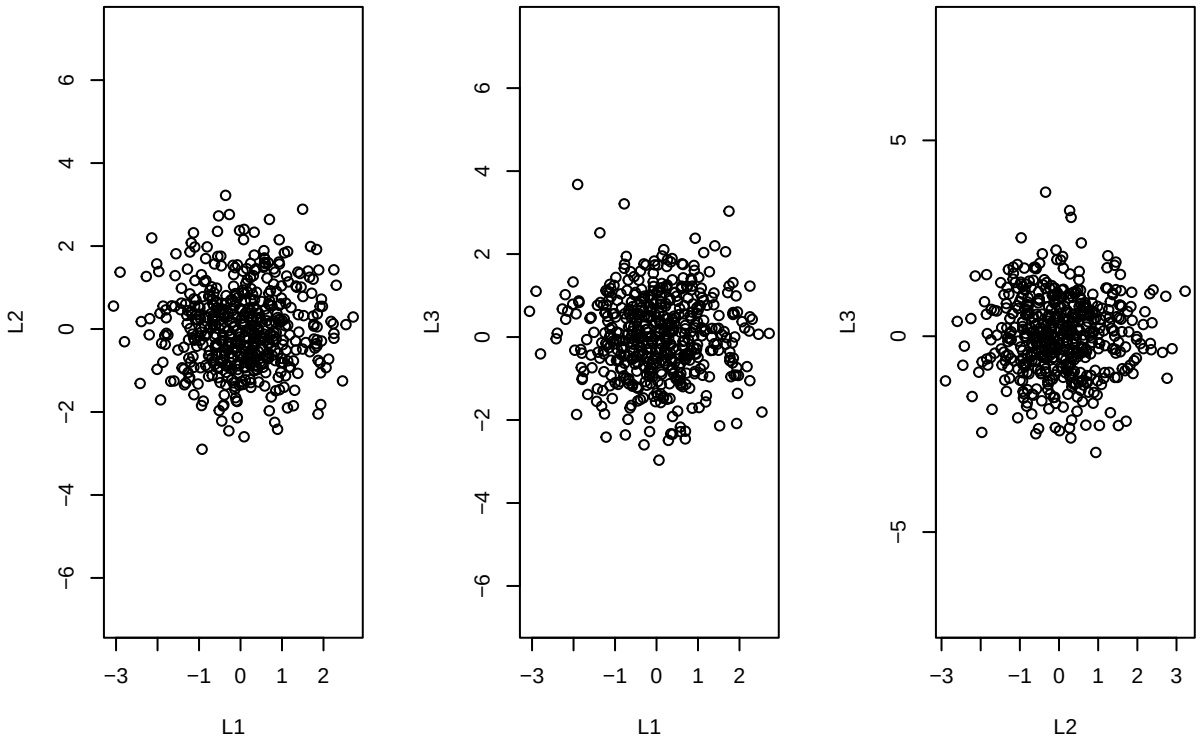
```
plot(PCA[,1],PCA[,2],xlab="L1",ylab="L2")
plot(PCA[,1],PCA[,3],xlab="L1",ylab="L3")
plot(PCA[,2],PCA[,3],xlab="L2",ylab="L3")
plot(PCAcor[,1],PCAcor[,2],xlab="L1",ylab="L2")
plot(PCAcor[,1],PCAcor[,3],xlab="L1",ylab="L3")
plot(PCAcor[,2],PCAcor[,3],xlab="L2",ylab="L3")
```




```
plot(ZCA[,1],ZCA[,2],xlab="L1",ylab="L2")
plot(ZCA[,1],ZCA[,3],xlab="L1",ylab="L3")
plot(ZCA[,2],ZCA[,3],xlab="L2",ylab="L3")
plot(ZCAcor[,1],ZCAcor[,2],xlab="L1",ylab="L2")
plot(ZCAcor[,1],ZCAcor[,3],xlab="L1",ylab="L3")
plot(ZCAcor[,2],ZCAcor[,3],xlab="L2",ylab="L3")
```




```
par(mfrow=c(1,3))
plot(Chole[,1],Chole[,2],xlab="L1",ylab="L2",asp=1)
plot(Chole[,1],Chole[,3],xlab="L1",ylab="L3",asp=1)
plot(Chole[,2],Chole[,3],xlab="L2",ylab="L3",asp=1)
```




```
par(mfrow=c(2,4))
plot(ZCA,ZCAcor,asp=1)
plot(ZCA,PCA,asp=1)
plot(ZCA,PCAcor,asp=1)
plot(ZCA,Chole,asp=1)
plot(PCA,PCAcor,asp=1)
plot(PCA,Chole,asp=1)
plot(PCAcor,Chole,asp=1)

cor(data,ZCA)
```

```
##              L1        L2         L3
## [1,] 0.98590777 0.1376002 0.09514232
## [2,] 0.11736575 0.7992103 0.58948130
## [3,] 0.06599621 0.4793943 0.87511461
```

```
cor(data,ZCAcor)
```

```
##              L1        L2         L3
## [1,] 0.98818969 0.1314869 0.07869146
## [2,] 0.13148686 0.8346348 0.53487931
## [3,] 0.07869146 0.5348793 0.84125607
```

```
cor(data,PCA)
```

```
##             L1          L2          L3
## [1,] 0.3396211 -0.94001327  0.03213319
## [2,] 0.9651543  0.04011608 -0.25858810
```

```
## [3,] 0.9778306  0.13090484  0.16343580
```

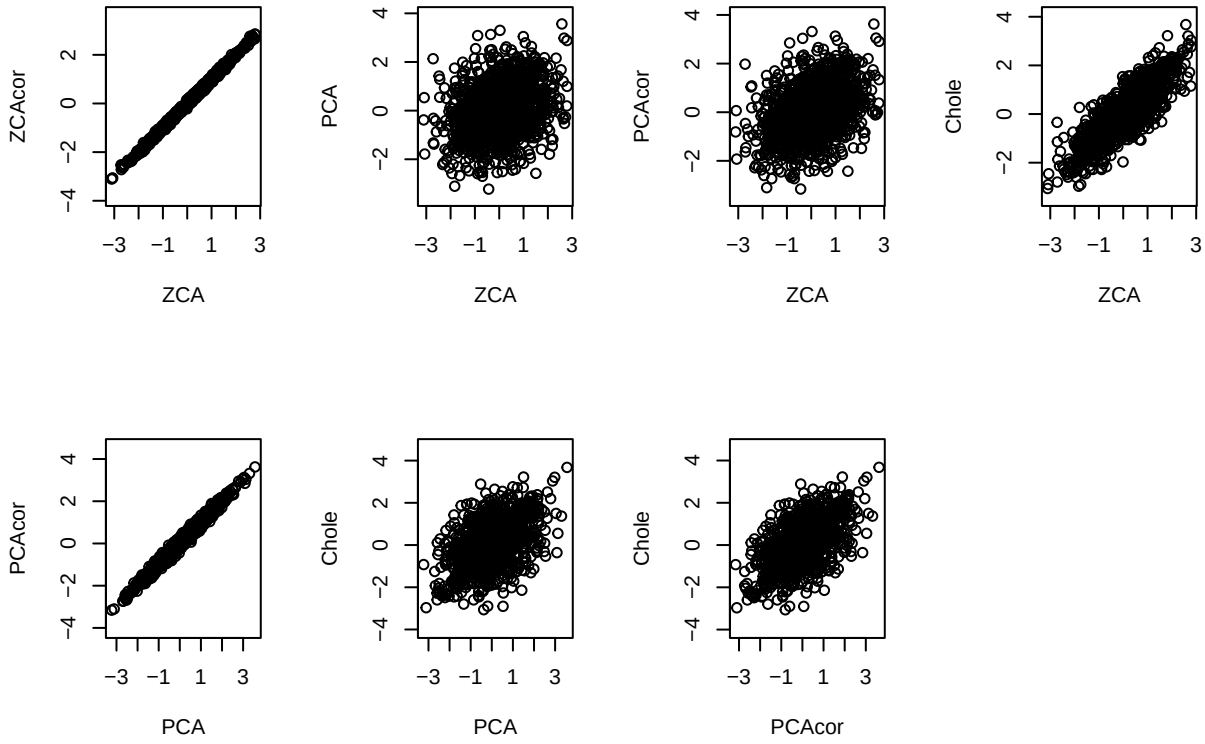
```
cor(data,PCAcor)
```

```
##             L1         L2          L3
## [1,] 0.4607066 -0.8873891  0.01702929
## [2,] 0.9604938  0.1770147 -0.21475000
## [3,] 0.9444154  0.2528597  0.21009878
```

```
cor(data,Chole)
```

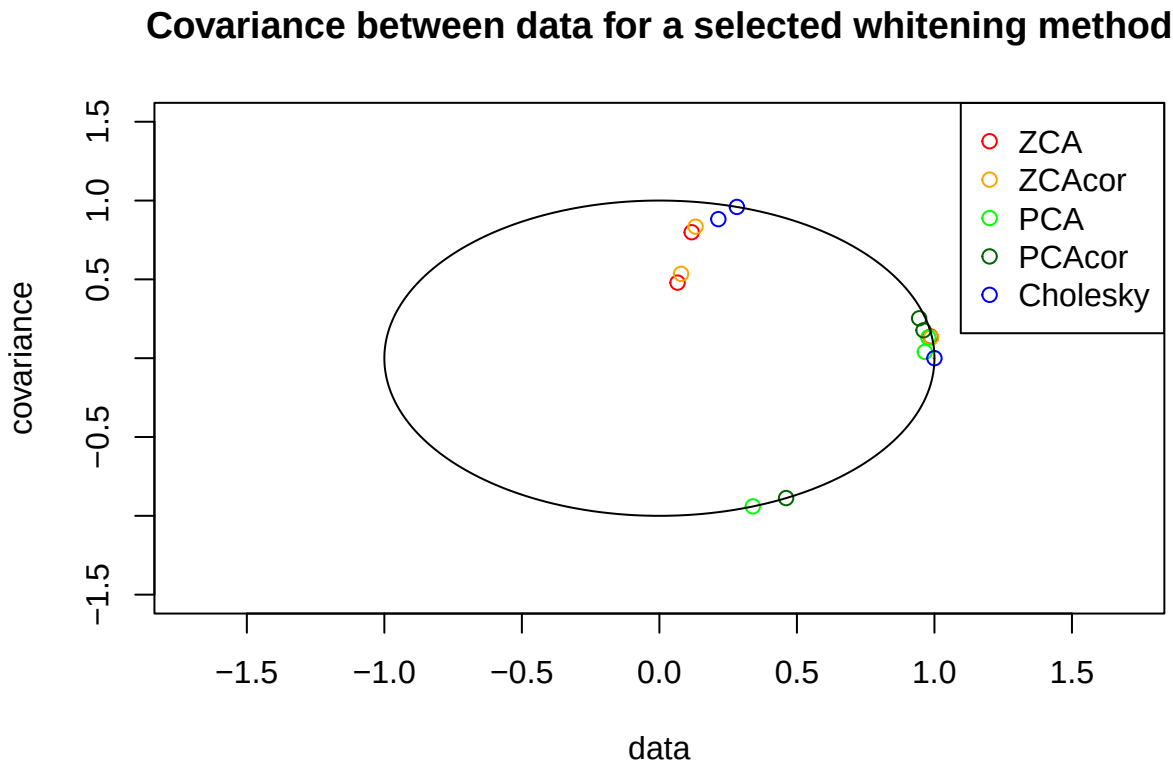
```
##             L1           L2            L3
## [1,] 1.0000000 6.055319e-16 -5.501107e-16
## [2,] 0.2817679 9.594826e-01 -1.413780e-15
## [3,] 0.2142914 8.821065e-01  4.194846e-01
```

```
par(mfrow=c(1,1))
```



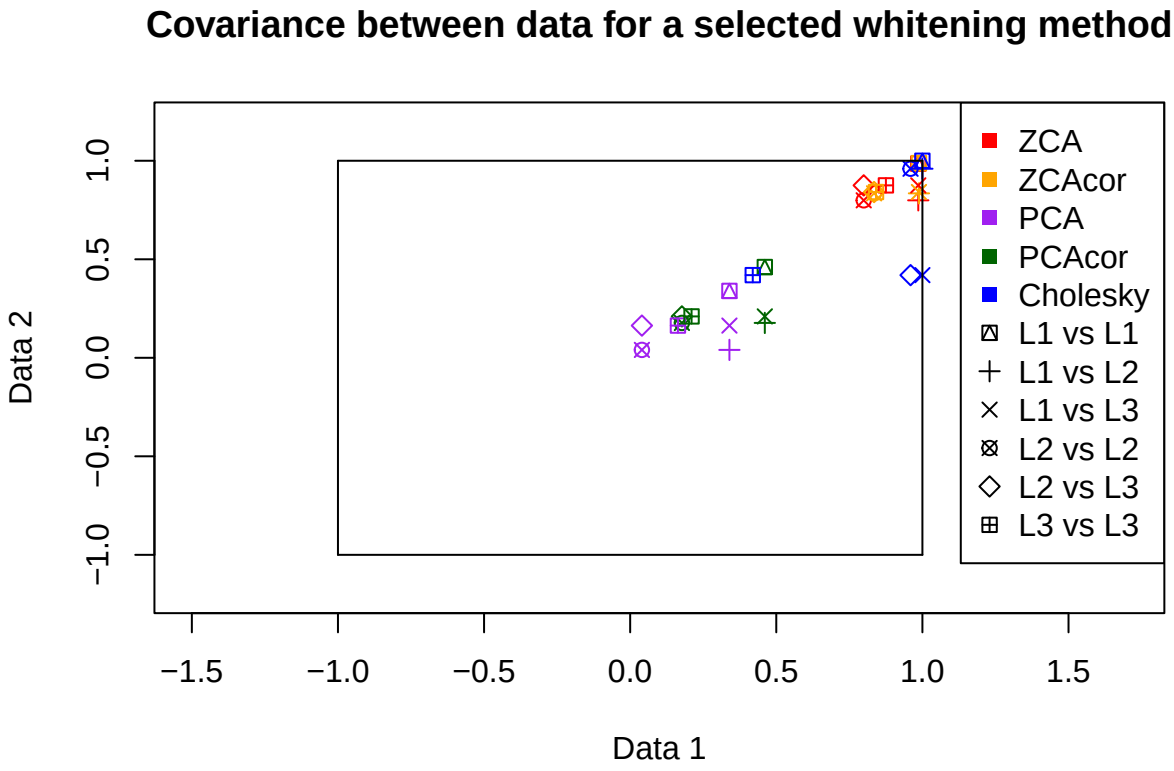

```
plot(cor(data,ZCA),col="red",
xlim=c(-1.7,1.7),ylim=c(-1.5,1.5),xlab="data",ylab="covariance",
main="Covariance between data for a selected whitening method")
points(cor(data,ZCAcor),col="orange")
points(cor(data,PCA),col="green")
points(cor(data,PCAcor),col="darkgreen")
points(cor(data,Chole),col="blue")
t<-seq(0,2*pi,by=0.01)
lines(cos(t),sin(t))
legend("topright",
```

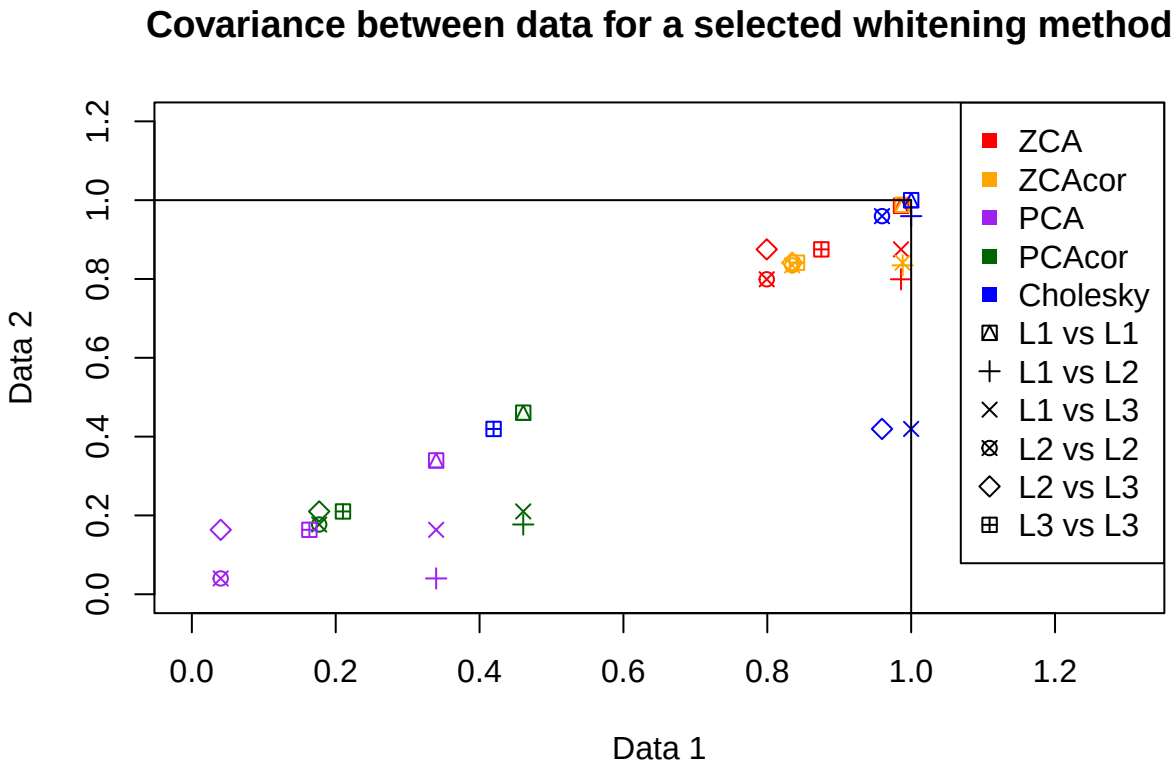
```
legend=c("ZCA","ZCAcor","PCA","PCAcor","Cholesky"),
col=c("red","orange","green","darkgreen","blue"),pch=1)
```

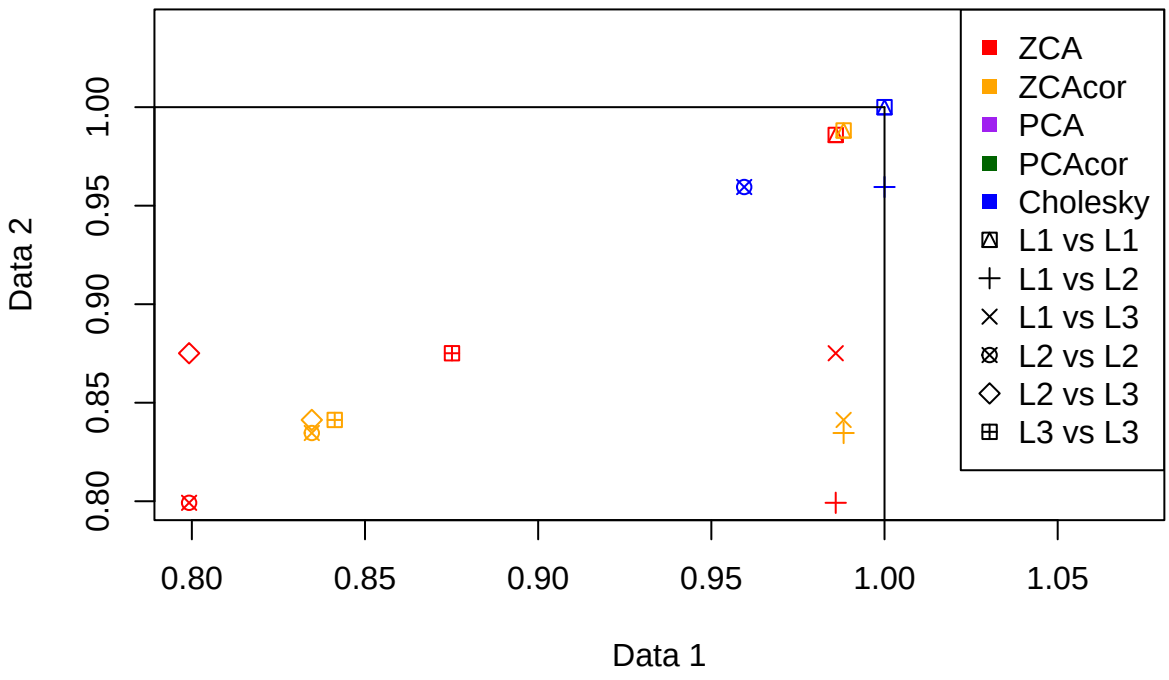



```
for (m in 1:3){
    if (m==1){
        plot(NA,xlim=c(-1.5,1.7),ylim=c(-1.2,1.2),
        xlab="Data 1",
        ylab="Data 2",
        main="Covariance between data for a selected whitening method")
    }
    if (m==2){
        plot(NA,xlim=c(0,1.3),ylim=c(0,1.2),
        xlab="Data 1",
        ylab="Data 2",
        main="Covariance between data for a selected whitening method")
    }
    if (m==3){
        plot(NA,xlim=c(0.8,1.07),ylim=c(0.8,1.04),
        xlab="Data 1",
        ylab="Data 2",
        main="Covariance between data for a selected whitening method")
    }
    for (i in 1:3){
        for (j in 1:3){
        if (i<=j){
            if (i!=j){
```

```
            #print(c(i,j,i+j))
            points(cor(data,ZCA)[i,i],cor(data,ZCA)[j,j],col="red",pch=i+j)
            points(cor(data,ZCAcor)[i,i],cor(data,ZCAcor)[j,j],col="orange",pch=i+j)
            points(cor(data,PCA)[i,i],cor(data,PCA)[j,j],col="purple",pch=i+j)
            points(cor(data,PCAcor)[i,i],cor(data,PCAcor)[j,j],col="darkgreen",pch=i+j)
            points(cor(data,Chole)[i,i],cor(data,Chole)[j,j],col="blue",pch=i+j)
        }
        if (i==j){
            #print(15-i)
            points(cor(data,ZCA)[i,i],cor(data,ZCA)[j,j],col="red",pch=15-i)
            points(cor(data,ZCAcor)[i,i],cor(data,ZCAcor)[j,j],col="orange",pch=15-i)
            points(cor(data,PCA)[i,i],cor(data,PCA)[j,j],col="purple",pch=15-i)
            points(cor(data,PCAcor)[i,i],cor(data,PCAcor)[j,j],col="darkgreen",pch=15-i)
            points(cor(data,Chole)[i,i],cor(data,Chole)[j,j],col="blue",pch=15-i)
        }
}}}
segments(-1,-1,-1,1)
segments(-1,-1,1,-1)
segments(1,-1,1,1)
segments(-1,1,1,1)
legend("topright",
legend=c("ZCA","ZCAcor","PCA",
"PCAcor","Cholesky",
"L1 vs L1","L1 vs L2","L1 vs L3",
"L2 vs L2","L2 vs L3","L3 vs L3"),
col=c("red","orange","purple","darkgreen","blue","black",
"black","black","black","black","black"),
pch=c(15,15,15,15,15,14,3,4,13,5,12))
}
```


Covariance between data for a selected whitening method
Data 2
1.0
0.5
0.0
−0.5
−1.0
−1.5
−1.0
−0.5
0.0
0.5
1.0
1.5
Data 1
ZCA
ZCAcor
PCA
PCAcor
Cholesky
L1 vs L1
L1 vs L2
L1 vs L3
L2 vs L2
L2 vs L3
L3 vs L3


Covariance between data for a selected whitening method
Data 2
0.0
0.2
0.4
0.6
0.8
1.0
1.2
Data 1
0.0
0.2
0.4
0.6
0.8
1.0
1.2
ZCA
ZCAcor
PCA
PCAcor
Cholesky
L1 vs L1
L1 vs L2
L1 vs L3
L2 vs L2
L2 vs L3
L3 vs L3




```
sum(diag(cor(data,ZCA)))/3
```

```
## [1] 0.8867442
```

```
sum(diag(cor(data,ZCAcor)))/3
```

```
## [1] 0.8880269
```

```
sum(diag(cor(data,PCA)))/3
```

```
## [1] 0.1810577
```

```
sum(diag(cor(data,PCAcor)))/3
```

```
## [1] 0.2826067
```

```
sum(diag(cor(data,Chole)))/3
```

```
## [1] 0.7929891
```

```
temp<-t(cor(data,ZCA))%*%cor(data,ZCA)
sum(diag(temp)^2)
```

```
## [1] 3.027729
```

```
temp<-t(cor(data,ZCAcor))%*%cor(data,ZCAcor)
sum(diag(temp)^2)
```

```
## [1] 3
```

```r
temp<-t(cor(data,PCA))%*%cor(data,PCA)
sum(diag(temp)^2)
```

```
## [1] 4.835305
```

```r
temp<-t(cor(data,PCAcor))%*%cor(data,PCAcor)
sum(diag(temp)^2)
```

```
## [1] 4.895005
```

```r
temp<-t(cor(data,Chole))%*%cor(data,Chole)
sum(diag(temp)^2)
```

```
## [1] 4.182941
```

We found that

1. ZCA-cor whitening produces the largest average componentwise cross-correlation.
2. PCA-cor whitening produces the largest compression in terms of sums of squared cross-correlation.

Example 5

Recall that

The whitening matrix is defined as

$$W = Q_1 \Sigma^{\frac{1}{2}} = Q_2 P^{\frac{1}{2}} V^{\frac{1}{2}}$$

where Q_1, and Q_2 are othogonal matrix, $P = V^{\frac{1}{2}} \Sigma V^{\frac{1}{2}}$, and V is the diagonal matrix containing the variances.

Cross covariance matrix Φ can be shown as

$$\Phi = \Sigma W^T$$

Similarly, the cross correlation matrix Ψ can be shown as

$$\Psi = V^{\frac{1}{2}} \Sigma W^T$$

In the following, we defined a function called "diagonalmatrix" which is use to find the matrix D^k.

$$A^k = P D^k P^{-1}$$

where D is the diagonal matrix of the eigenvalues of A, and the columns of P are the corresponding eigenvectors. If P is orthogonal, then $P^{-1} = P^T$. This is called the eigendecomposition of A.

```r
diagonalmatrix<-function(matrix1,power){
    d<-nrow(matrix1)
    sigmapower<-matrix(0,nrow=d,ncol=d)
    eigenvalue<-eigen(matrix1)$values
    for (i in 1:d){
        for (j in 1:d){
            if (i==j){
                sigmapower[i,j]<-eigenvalue[i]^power
            }
            else{
                sigmapower[i,j]<-0
            }
        }
    }
    return(sigmapower)
}
```

```r
tester1<-matrix(c(1,0,0,1),nrow=2)
diagonalmatrix(tester1,1/2)
```

```
##      [,1] [,2]
## [1,]    1    0
## [2,]    0    1
```

```r
tester2<-matrix(c(4,1,1,3),nrow=2)
diagonalmatrix(tester2,1/2)
```

```
##          [,1]     [,2]
## [1,] 2.148961 0.000000
## [2,] 0.000000 1.543362
```

```r
tester3<-matrix(c(4,1,1,1,5,1,1,1,10),nrow=3)
diagonalmatrix(tester3,1/2)
```

```
##          [,1]     [,2]     [,3]
## [1,] 3.226504 0.000000 0.000000
## [2,] 0.000000 2.286027 0.000000
## [3,] 0.000000 0.000000 1.834053
```

```r
# Finding the diagonal matrix whose entries are each variance raised to the kth power
findingV<-function(matrix1,power){
    d<-nrow(matrix1)
    sigmapower<-matrix(0,nrow=d,ncol=d)
    for (i in 1:d){
        for (j in 1:d){
            if (i==j){
                sigmapower[i,j]<-matrix1[i,j]^power
            }
            else{
                sigmapower[i,j]<-0
            }
        }
    }
    return(sigmapower)
}
tester1<-matrix(c(1,0,0,1),nrow=2)
findingV(tester1,1/2)
```

```
##      [,1] [,2]
## [1,]    1    0
## [2,]    0    1
```

```r
tester2<-matrix(c(4,1,1,3),nrow=2)
findingV(tester2,1)
```

```
##      [,1] [,2]
## [1,]    4    0
## [2,]    0    3
```

```r
tester3<-matrix(c(4,1,1,1,5,1,1,1,10),nrow=3)
findingV(tester3,1/2)
```

```
##      [,1]     [,2]     [,3]
## [1,]    2 0.000000 0.000000
```

```
## [2,]    0 2.236068 0.000000
## [3,]    0 0.000000 3.162278
```

```
# Finding a matrix A^k, for some k in Z.
findAkpower<-function(Matrix1,power){
    eigenvector<-eigen(Matrix1)$vectors
    diagon<-diagonalmatrix(Matrix1,power)
    return(eigenvector%*%diagon %*% t(eigenvector))
}
tester1<-matrix(c(1,0,0,1),nrow=2)
findAkpower(tester1,10)
```

```
##      [,1] [,2]
## [1,]    1    0
## [2,]    0    1
```

```
tester2<-matrix(c(4,1,1,3),nrow=2)
findAkpower(tester2,4)
```

```
##      [,1] [,2]
## [1,]  338  189
## [2,]  189  149
```

```
tester3<-matrix(c(4,1,1,1,5,1,1,1,10),nrow=3)
findAkpower(tester3,1/2)
```

```
##           [,1]      [,2]      [,3]
## [1,] 1.9778648 0.2304431 0.1869406
## [2,] 0.2304431 2.2170084 0.1782400
## [3,] 0.1869406 0.1782400 3.1517112
```

```
# Finding the correlation matrix P.
findingP<-function(Sigma,power){
    temp<-findingV(Sigma,-1/2)
    P<-temp%*%Sigma%*%temp
    return(findAkpower(P,power))
}

# Finding the ZCA whitening matrix.
ZCA<-function(Sigma){
    eigenvector<-eigen(Sigma)$vectors
    temp<-diagonalmatrix(Sigma,-1/2)
    temp2<-eigenvector%*%temp%*%t(eigenvector)
    return(temp2)
}
tester1<-matrix(c(1,0,0,1),nrow=2)
ZCA(tester1)
```

```
##      [,1] [,2]
## [1,]    1    0
## [2,]    0    1
```

```
tester2<-matrix(c(4,1,1,3),nrow=2)
ZCA(tester2)
```

```
##             [,1]        [,2]
## [1,]  0.51580915 -0.08165898
## [2,] -0.08165898  0.59746814
```

```r
tester3<-matrix(c(4,1,1,1,5,1,1,1,10),nrow=3)
ZCA(tester3)
```

```
##             [,1]        [,2]        [,3]
## [1,]  0.51417282 -0.05122581 -0.02760064
## [2,] -0.05122581  0.45822197 -0.02287561
## [3,] -0.02760064 -0.02287561  0.32021875
```

```r
# Finding the PCA whitening matrix.
PCA<-function(Sigma){
    hat<-diagonalmatrix(Sigma,-1/2)
    eigenvector<-eigen(Sigma)$vectors
    temp<- hat%*%t(eigenvector)
    return(temp)
}

tester1<-matrix(c(1,0,0,1),nrow=2)
PCA(tester1)
```

```
##      [,1] [,2]
## [1,]    0    1
## [2,]   -1    0
```

```r
tester2<-matrix(c(4,1,1,3),nrow=2)
PCA(tester2)
```

```
##            [,1]       [,2]
## [1,] -0.3958428 -0.2446443
## [2,]  0.3406402 -0.5511674
```

```r
tester3<-matrix(c(4,1,1,1,5,1,1,1,10),nrow=3)
PCA(tester3)
```

```
##             [,1]        [,2]       [,3]
## [1,] -0.05663835 -0.06547384 -0.2975966
## [2,] -0.20297437 -0.36854323  0.1197126
## [3,]  0.47260244 -0.27019219 -0.0305007
```

```r
# Finding ZCA-cor whitening matrix.
ZCAcor<-function(Sigma){
    temp1<- findingP(Sigma,-1/2)
    temp2<- findingV(Sigma,-1/2)
    return(temp1%*%temp2)
}

tester1<-matrix(c(1,0,0,1),nrow=2)
ZCAcor(tester1)
```

```
##      [,1] [,2]
## [1,]    1    0
## [2,]    0    1
```

```r
tester2<-matrix(c(4,1,1,3),nrow=2)
ZCAcor(tester2)
```

```
##             [,1]        [,2]
## [1,]  0.51664483 -0.08798026
## [2,] -0.07619314  0.59657006
```

```r
tester3<-matrix(c(4,1,1,1,5,1,1,1,10),nrow=3)
ZCAcor(tester3)
```

```
##             [,1]        [,2]        [,3]
## [1,]  0.51333507 -0.04888781 -0.02244237
## [2,] -0.05465823  0.45824573 -0.01917796
## [3,] -0.03548451 -0.02712173  0.32086396
```

```r
# Finding PCA-cor whitening matrix.
PCAcor<-function(Matrix){
    P1<- findingP(Matrix,1)
    G1<-eigen(P1)$vectors
    the<-diagonalmatrix(P1,-1/2)
    V1<-findingV(Matrix,-1/2)
    temp<- the %*% t(G1) %*% V1
    return(temp)
}

tester1<-matrix(c(1,0,0,1),nrow=2)
PCAcor(tester1)
```

```
##      [,1] [,2]
## [1,]    0    1
## [2,]   -1    0
```

```r
tester2<-matrix(c(4,1,1,3),nrow=2)
PCAcor(tester2)
```

```
##            [,1]       [,2]
## [1,] -0.3114464 -0.3596273
## [2,]  0.4191997 -0.4840502
```

```r
tester3<-matrix(c(4,1,1,1,5,1,1,1,10),nrow=3)
PCAcor(tester3)
```

```
##            [,1]       [,2]       [,3]
## [1,] -0.2636350 -0.2300338 -0.1404987
## [2,] -0.1555162 -0.2092144  0.2879956
## [3,]  0.4172180 -0.3412162 -0.0338200
```

```r
chole<-function(Matrix){
    upper<-chol(Matrix)
    lower<-t(upper)
    return(solve(lower))
}

tester1<-matrix(c(1,0,0,1),nrow=2)
chole(tester1)
```

```
##      [,1] [,2]
## [1,]    1    0
## [2,]    0    1
```

```r
solve(t(chole(tester1)) %*% chole(tester1))
```

```
##      [,1] [,2]
## [1,]    1    0
## [2,]    0    1
```

```
tester2<-matrix(c(4,1,1,3),nrow=2)
chole(tester2)
```

```
##            [,1]      [,2]
## [1,]  0.5000000 0.0000000
## [2,] -0.1507557 0.6030227
```

```
solve(t(chole(tester2)) %*% chole(tester2))
```

```
##      [,1] [,2]
## [1,]    4    1
## [2,]    1    3
```

```
tester3<-matrix(c(4,1,1,1,5,1,1,1,10),nrow=3)
chole(tester3)
```

```
##            [,1]       [,2]      [,3]
## [1,]  0.5000000  0.0000000 0.0000000
## [2,] -0.1147079  0.4588315 0.0000000
## [3,] -0.0678356 -0.0508767 0.3222191
```

```
solve(t(chole(tester3)) %*% chole(tester3))
```

```
##      [,1] [,2] [,3]
## [1,]    4    1    1
## [2,]    1    5    1
## [3,]    1    1   10
```

```
crosscov<-function(Sigma,method="ZCA"){
    if (method=="ZCA"){
        W<-ZCA(Sigma)
    }
    if (method=="PCA"){
        W<-PCA(Sigma)
    }
    if (method=="ZCAcor"){
        W<-ZCAcor(Sigma)
    }
    if (method=="PCAcor"){
        W<-PCAcor(Sigma)
    }
    if (method=="Cholesky"){
        W<-chole(Sigma)
    }
    temp<-Sigma %*% t(W)
    return(temp)
}
tester1<-matrix(c(1,0,0,1),nrow=2)
crosscov(tester1,"ZCA")
```

```
##      [,1] [,2]
## [1,]    1    0
## [2,]    0    1
```

```
tester2<-matrix(c(4,1,1,3),nrow=2)
crosscov(tester2,"PCA")
```

```
##           [,1]       [,2]
```

```
## [1,] -1.828016  0.8113934
## [2,] -1.129776 -1.3128621
```

```
tester3<-matrix(c(4,1,1,1,5,1,1,1,10),nrow=3)
crosscov(tester3)
```

```
##           [,1]      [,2]      [,3]
## [1,] 1.9778648 0.2304431 0.1869406
## [2,] 0.2304431 2.2170084 0.1782400
## [3,] 0.1869406 0.1782400 3.1517112
```

```
tester4<-matrix(c(4,1,1,3),nrow=2)
method1<-c("ZCA","PCA","ZCAcor","PCAcor","Cholesky")
p<-matrix(0,nrow=5,ncol=4)
for (i in 1:5){
    p[i,]<-as.vector(crosscov(tester4,method1[i]))
}
print(p)
```

```
##           [,1]       [,2]      [,3]      [,4]
## [1,]  1.981578  0.2708322 0.2708322  1.710745
## [2,] -1.828016 -1.1297757 0.8113934 -1.312862
## [3,]  1.978599  0.2527041 0.2917975  1.713517
## [4,] -1.605413 -1.3903283 1.1927488 -1.032951
## [5,]  2.000000  0.5000000 0.0000000  1.658312
```

```
crosscorr<-function(Sigma,method="ZCA"){
    if (method=="ZCA"){
        W<-ZCA(Sigma)
    }
    if (method=="PCA"){
        W<-PCA(Sigma)
    }
    if (method=="ZCAcor"){
        W<-ZCAcor(Sigma)
    }
    if (method=="PCAcor"){
        W<-PCAcor(Sigma)
    }
    if (method=="Cholesky"){
        W<-chole(Sigma)
    }
    temp<-findingV(Sigma,-1/2) %*% Sigma %*% t(W)
    return(temp)
}
tester1<-matrix(c(1,0,0,1),nrow=2)
crosscorr(tester1,"ZCA")
```

```
##      [,1] [,2]
## [1,]    1    0
## [2,]    0    1
```

```
tester2<-matrix(c(4,1,1,3),nrow=2)
crosscorr(tester2,"PCA")
```

```
##           [,1]       [,2]
## [1,] -0.9140078  0.4056967
```

```
## [2,] -0.6522763 -0.7579813
```

```
tester3<-matrix(c(4,1,1,1,5,1,1,1,10),nrow=3)
crosscorr(tester3)
```

```
##            [,1]       [,2]       [,3]
## [1,] 0.98893240 0.11522156 0.09347028
## [2,] 0.10305730 0.99147632 0.07971137
## [3,] 0.05911579 0.05636445 0.99665860
```

```
tester4<-matrix(c(4,1,1,3),nrow=2)
method1<-c("ZCA","PCA","ZCAcor","PCAcor","Cholesky")
p<-matrix(0,nrow=5,ncol=4)
for (i in 1:5){
    p[i,]<-as.vector(crosscorr(tester4,method1[i]))
}
print(p)
```

```
##            [,1]       [,2]      [,3]       [,4]
## [1,]  0.9907888  0.1563650 0.1354161  0.9876993
## [2,] -0.9140078 -0.6522763 0.4056967 -0.7579813
## [3,]  0.9892995  0.1458988 0.1458988  0.9892995
## [4,] -0.8027064 -0.8027064 0.5963744 -0.5963744
## [5,]  1.0000000  0.2886751 0.0000000  0.9574271
```

```
# We are now interested in the proportion of the total variance across different whitening methods.
# The relative contribution of each element in z to explain the total variation.
ZCAprop<-function(Sigma){
    empty<-matrix(0,nrow=nrow(Sigma),ncol=1)
    sumvar<-sum(diag(Sigma))
    for (i in 1:nrow(Sigma)){
        empty[i]<-Sigma[i,i]/sumvar
    }
    return(empty)
}
tester1<-matrix(c(1,0,0,1),nrow=2)
ZCAprop(tester1)
```

```
##      [,1]
## [1,]  0.5
## [2,]  0.5
```

```
tester2<-matrix(c(4,1,1,3),nrow=2)
ZCAprop(tester2)
```

```
##           [,1]
## [1,] 0.5714286
## [2,] 0.4285714
```

```
tester3<-matrix(c(4,1,1,1,5,1,1,1,10),nrow=3)
ZCAprop(tester3)
```

```
##           [,1]
## [1,] 0.2105263
## [2,] 0.2631579
## [3,] 0.5263158
```

```
ZCAcorprop<-function(Sigma){
    empty<-matrix(0,nrow=nrow(Sigma),ncol=1)
    for (i in 1:nrow(Sigma)){
        empty[i]<-nrow(Sigma)
    }
    return(empty)
}
tester1<-matrix(c(1,0,0,1),nrow=2)
ZCAcorprop(tester1)
```

```
##      [,1]
## [1,]    2
## [2,]    2
```

```
tester2<-matrix(c(4,1,1,3),nrow=2)
ZCAcorprop(tester2)
```

```
##      [,1]
## [1,]    2
## [2,]    2
```

```
tester3<-matrix(c(4,1,1,1,5,1,1,1,10),nrow=3)
ZCAcorprop(tester3)
```

```
##      [,1]
## [1,]    3
## [2,]    3
## [3,]    3
```

```
PCAprop<-function(Sigma){
    eigenvalue<-eigen(Sigma)$values
    empty<-matrix(0,nrow=nrow(Sigma),ncol=1)
    for (i in 1:nrow(Sigma)){
        empty[i]<-eigenvalue[i]/sum(eigenvalue)
    }
    return(empty)
}
tester1<-matrix(c(1,0,0,1),nrow=2)
PCAprop(tester1)
```

```
##      [,1]
## [1,]  0.5
## [2,]  0.5
```

```
tester2<-matrix(c(4,1,1,3),nrow=2)
PCAprop(tester2)
```

```
##           [,1]
## [1,] 0.6597191
## [2,] 0.3402809
```

```
tester3<-matrix(c(4,1,1,1,5,1,1,1,10),nrow=3)
PCAprop(tester3)
```

```
##           [,1]
## [1,] 0.5479120
## [2,] 0.2750485
```

```
## [3,] 0.1770395
```

```
PCAcorprop<-function(Sigma){
    temp<-findingP(Sigma,1)
    eigenvalue<-eigen(temp)$values
    empty<-matrix(0,nrow=nrow(Sigma),ncol=1)
    for (i in 1:nrow(Sigma)){
        empty[i]<-eigenvalue[i]/sum(eigenvalue)
    }
    return(empty)
}
tester1<-matrix(c(1,0,0,1),nrow=2)
PCAcorprop(tester1)
```

```
##      [,1]
## [1,]  0.5
## [2,]  0.5
```

```
tester2<-matrix(c(4,1,1,3),nrow=2)
PCAcorprop(tester2)
```

```
##           [,1]
## [1,] 0.6443376
## [2,] 0.3556624
```

```
tester3<-matrix(c(4,1,1,1,5,1,1,1,10),nrow=3)
PCAcorprop(tester3)
```

```
##           [,1]
## [1,] 0.4504563
## [2,] 0.2911185
## [3,] 0.2584252
```